

Assignment – 18.2

Name: Alli Harika

Batch:21

Ht.no:2303A510I7

Task Description – 1 (Air Quality API Integration)

- Task:

Use AI assistance to develop a Python program that retrieves real-time air quality information for a specified city.

Instructions:

- Generate Python code using the requests library.
- Handle errors such as invalid city names, missing or incorrect API keys, and network timeouts.
- Extract and display AQI value, pollution level, and dominant pollutant in a readable format.

Sample Code:

```
import requests  
city = "Delhi"  
url = "https://api.example.com/airquality"
```

Expected Output:

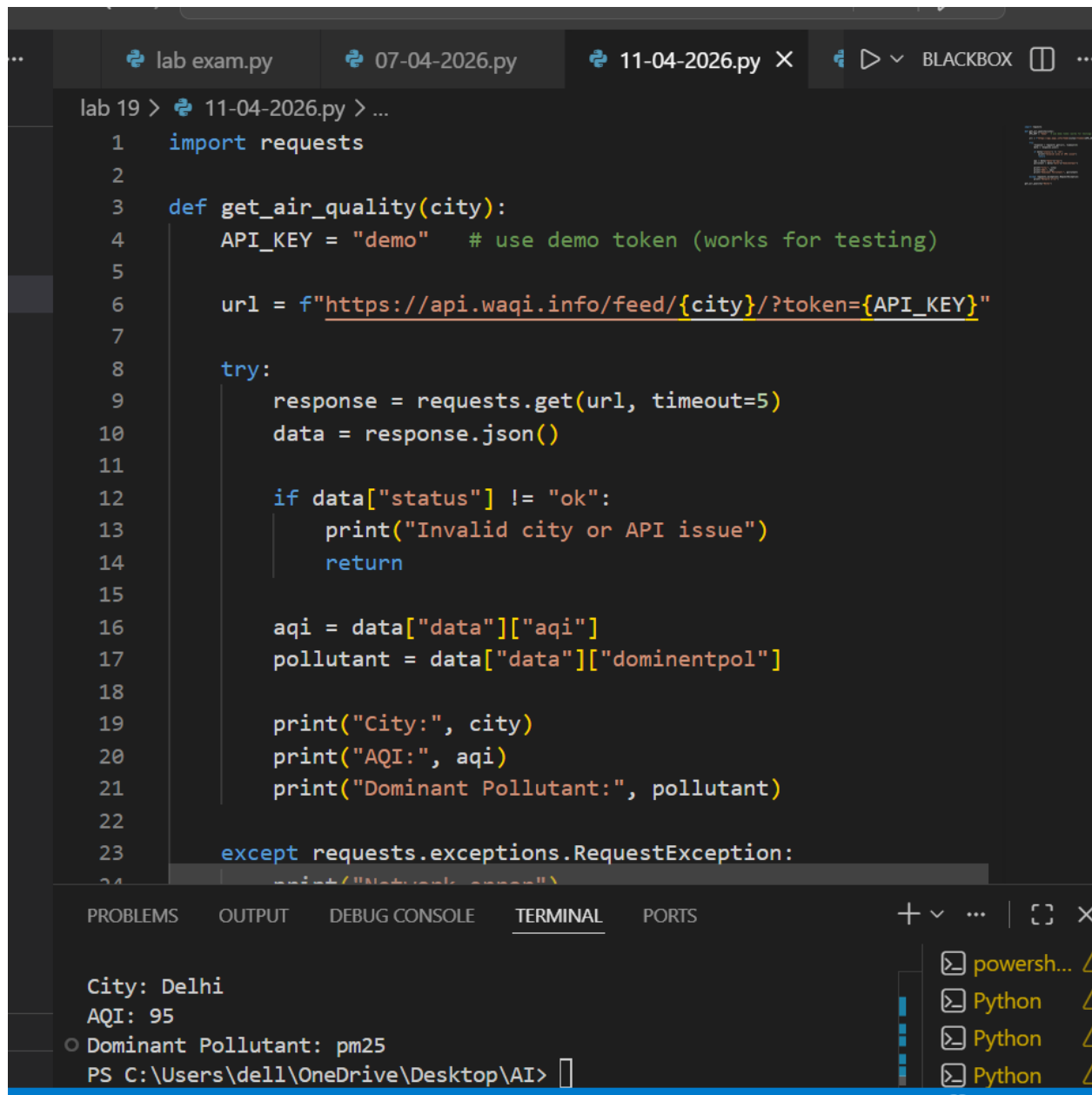
- A Python script that successfully fetches and displays air quality data with proper error handling.

Prompt:

Generate Python code using the requests library to fetch real-time air quality data for a given city.

Include error handling for invalid city names, API key issues, and network failures.

Display AQI, pollution level, and dominant pollutant.

The image shows a screenshot of a Visual Studio Code editor window. At the top, there are three tabs: 'lab exam.py', '07-04-2026.py', and '11-04-2026.py'. The active tab is '11-04-2026.py'. The editor displays a Python script with the following code:

```
lab 19 > 11-04-2026.py > ...
1 import requests
2
3 def get_air_quality(city):
4     API_KEY = "demo" # use demo token (works for testing)
5
6     url = f"https://api.waqi.info/feed/{city}/?token={API_KEY}"
7
8     try:
9         response = requests.get(url, timeout=5)
10        data = response.json()
11
12        if data["status"] != "ok":
13            print("Invalid city or API issue")
14            return
15
16        aqi = data["data"]["aqi"]
17        pollutant = data["data"]["dominantpol"]
18
19        print("City:", city)
20        print("AQI:", aqi)
21        print("Dominant Pollutant:", pollutant)
22
23    except requests.exceptions.RequestException:
24        print("Network error")
```

The bottom of the editor shows a terminal window with the following output:

```
City: Delhi
AQI: 95
Dominant Pollutant: pm25
PS C:\Users\dell\OneDrive\Desktop\AI>
```

Explanation:

This project retrieves **real-time air quality data** for a given city using an external API.

- The program sends an HTTP GET request to the API.
- The API responds with data in **JSON format**.
- From the response, important values like:
 - AQI (Air Quality Index)
 - Dominant pollutantare extracted and displayed.

Task Description – 2 (Language Translation API)

- Task:

Integrate a Language Translation API to translate text from English to two other languages.

Instructions:

- Use AI to generate API request code for translation.
- Validate user input text and target language codes.
- Handle API response errors and service unavailability.
- Display original and translated text clearly.

Sample Code:

```
import requests  
text = "Welcome to AI Assisted Coding"  
target_lang = "fr"
```

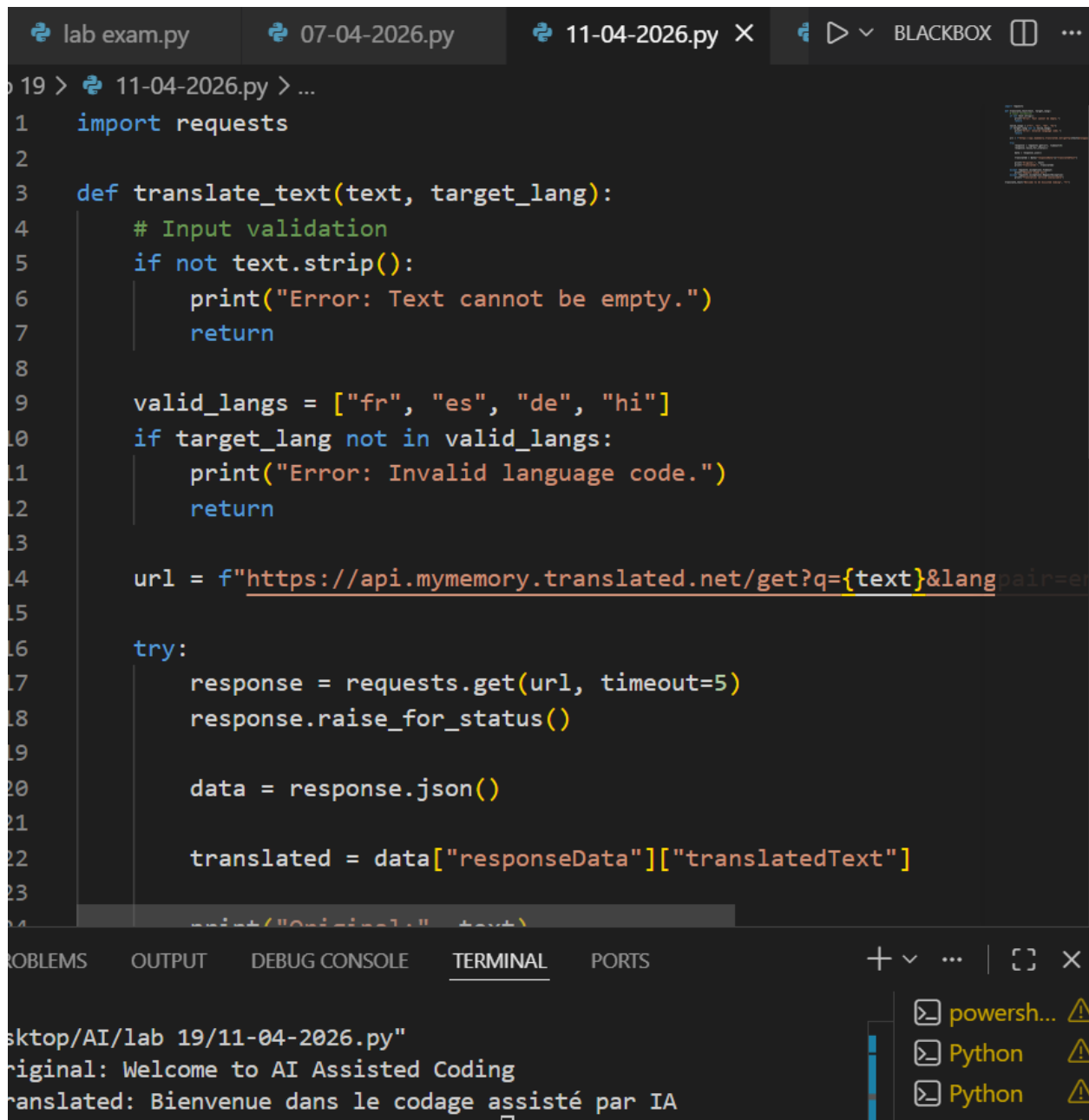
Expected Output:

- A working Python script that translates user input text with graceful error handling.

Prompt:

Generate Python code to translate English text into another language using an API.

Validate input and handle API errors. Display original and translated text.



```
lab exam.py 07-04-2026.py 11-04-2026.py X BLACKBOX
```

```
19 > 11-04-2026.py > ...
1  import requests
2
3  def translate_text(text, target_lang):
4      # Input validation
5      if not text.strip():
6          print("Error: Text cannot be empty.")
7          return
8
9      valid_langs = ["fr", "es", "de", "hi"]
10     if target_lang not in valid_langs:
11         print("Error: Invalid language code.")
12         return
13
14     url = f"https://api.mymemory.translated.net/get?q={text}&langpair=en"
15
16     try:
17         response = requests.get(url, timeout=5)
18         response.raise_for_status()
19
20         data = response.json()
21
22         translated = data["responseData"]["translatedText"]
23
24         print(f"Original: {text}")
25         print(f"Translated: {translated}")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
sktop/AI/lab 19/11-04-2026.py"
Original: Welcome to AI Assisted Coding
Translated: Bienvenue dans le codage assisté par IA
```

powerh... Python Python

Explanation

This project translates English text into another language using a translation API.

- User provides input text and target language code.
- The program sends a request to the API.
- The API returns translated text.
- Both original and translated text are displayed.

Task Description – 3 (Public Book Information API)

- Task:

Connect to a public Books API to fetch information about a book using its title or ISBN.

Instructions:

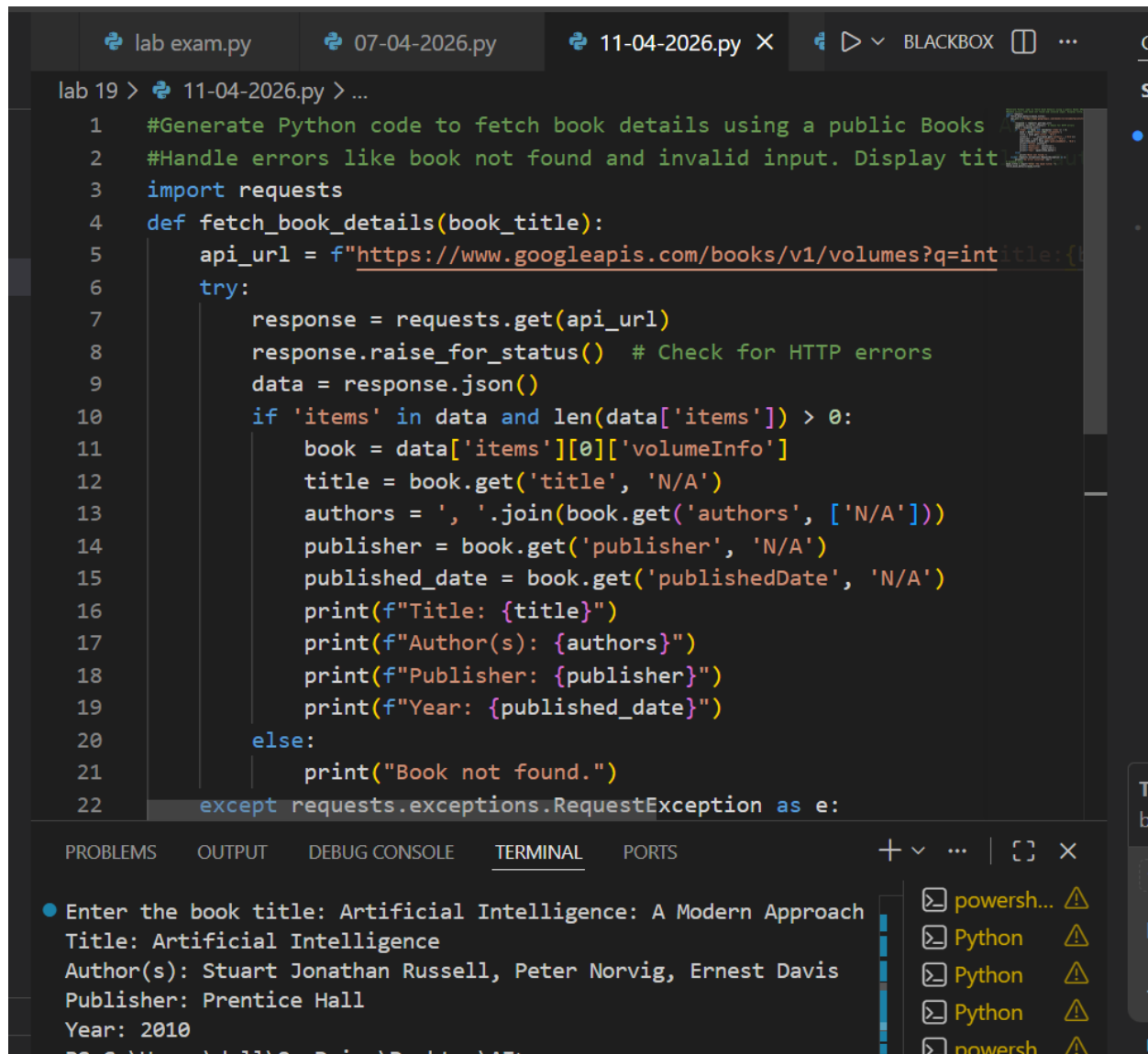
- Prompt AI to write Python code for sending GET requests to the API.
- Handle errors such as book not found, invalid input, and rate limits.
- Display book title, author, publisher, and publication year.

Sample Code:

```
import requests  
query = "Artificial Intelligence"
```

Expected Output:

- A Python program that retrieves and displays book details with robust error handling.



The screenshot shows a VS Code editor with a file named '11-04-2026.py' open. The code is a Python script that uses the 'requests' library to fetch book details from a public Books API. The script defines a function 'fetch_book_details' that takes a book title as input. It constructs an API URL and sends a GET request. If the request is successful, it extracts the book's title, authors, publisher, and published date, and prints them. If the book is not found, it prints 'Book not found.' The script also includes a try-except block to handle 'RequestException'.

```
lab 19 > 11-04-2026.py > ...
1 #Generate Python code to fetch book details using a public Books API
2 #Handle errors like book not found and invalid input. Display title, author, publisher, year
3 import requests
4 def fetch_book_details(book_title):
5     api_url = f"https://www.googleapis.com/books/v1/volumes?q=intitle:{book_title}"
6     try:
7         response = requests.get(api_url)
8         response.raise_for_status() # Check for HTTP errors
9         data = response.json()
10        if 'items' in data and len(data['items']) > 0:
11            book = data['items'][0]['volumeInfo']
12            title = book.get('title', 'N/A')
13            authors = ', '.join(book.get('authors', ['N/A']))
14            publisher = book.get('publisher', 'N/A')
15            published_date = book.get('publishedDate', 'N/A')
16            print(f>Title: {title}")
17            print(f>Author(s): {authors}")
18            print(f>Publisher: {publisher}")
19            print(f>Year: {published_date}")
20        else:
21            print("Book not found.")
22    except requests.exceptions.RequestException as e:
```

The terminal output shows the execution of the script. It prompts the user to enter a book title, and the user enters 'Artificial Intelligence: A Modern Approach'. The script then prints the book's details: Title, Author(s), Publisher, and Year.

```
● Enter the book title: Artificial Intelligence: A Modern Approach
Title: Artificial Intelligence
Author(s): Stuart Jonathan Russell, Peter Norvig, Ernest Davis
Publisher: Prentice Hall
Year: 2010
```

Prompt:

**Generate Python code to fetch book details using a public Books API.
Handle errors like book not found and invalid input. Display title, author,
publisher, year.**

Explanation

This project fetches book details using a public Books API.

- User enters a book title or keyword.
- The program sends a GET request to the API.
- It retrieves:
 - Title
 - Author

- Publisher
- Publication year

Task Description – 4 (Real-Time Application: Job Listings Aggregator)

- Scenario:

Develop a Job Listings Aggregator using a public Jobs API.

Requirements:

- Fetch the latest job postings for a given role or location.
- Handle errors such as invalid search terms, missing API keys, and request failures.
- Display job title, company name, location, and posting date in a numbered list.
- Implement a retry mechanism for failed API requests.

Sample Code:

```
import requests
```

```
keyword = "Python Developer"
```

Expected Output:

- A functional Python script that retrieves and displays job listings with proper error handling.

Prompt:

Generate Python code to fetch job listings using an API.

Display job title, company, location, and date.

Include retry mechanism and error handling.

9 > 11-04-2026.py > ...

```
import requests
import time

def fetch_jobs(keyword, retries=3, delay=3):
    url = f"https://remotive.com/api/remote-jobs?search={keyword}"

    for attempt in range(retries):
        try:
            response = requests.get(url, timeout=5)
            response.raise_for_status()

            data = response.json()

            # DEBUG: show total jobs count
            print("Total jobs found:", len(data.get("jobs", [])))

            jobs = data.get("jobs", [])

            if not jobs:
                print("No jobs found. Try different keyword.")
                return

            print(f"\nTop Jobs for '{keyword}':\n")

            for i, job in enumerate(jobs[:5], 1):
                print(f"{i}. {job['title']}")
```



```
lab 19 > 11-04-2026.py > ...
1 import requests

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

● Total jobs found: 4

Top Jobs for 'python':

1. Fullstack Developer (US - ET)
   Company: Chooose
   Location: USA
   Date: 2026-04-08T12:36:43

2. Technical Support Operator
   Company: KnownHost
   Location: USA
   Date: 2026-03-27T09:51:22

3. Tech Lead Databricks Data Engineer
   Company: Mitre Media
   Location: USA, Canada, USA timezones
   Date: 2026-03-14T20:46:28

4. Full-Stack Developer
   Company: ELECTE S.R.L.
   Location: Worldwide
   Date: 2026-03-13T15:17:31

○ PS C:\Users\dell\OneDrive\Desktop\AI> 
```

Explanation

This project collects job listings using a public Jobs API.

- User provides a keyword (e.g., "python").
- The program fetches job data from API.
- Displays:

- **Job title**
- **Company name**
- **Location**
- **Posting date**

Task Description – 5 Stock Market Data API Integration

- **Task:**

Use AI assistance to develop a Python program that retrieves real-time stock market data for a selected company.

Instructions:

- Generate Python code using the requests library to connect to a public Stock Market API.
- Validate user input for stock symbol.
- Handle common errors such as invalid symbols, API rate limits, missing API keys, and network failures.
- Extract and display stock name, current price, day high, day low, and percentage change.

Sample Code:

```
import requests  
symbol = "AAPL"
```

- **Expected Output:**

A Python script that successfully fetches and displays stock market information with robust error handling.

Prompt:

Generate Python code to fetch real-time stock data using an API.

Validate stock symbol and handle errors. Display price, high, low, and change.

```
lab 19 > 11-04-2026.py > ...
1 import requests
2
3 def get_stock(symbol):
4     url = f"https://stooq.com/q/1/?s={symbol}&f=sd2t2ohlcv&h&e=cs"
5
6     try:
7         response = requests.get(url, timeout=5)
8         response.raise_for_status()
9
10        data = response.text.split("\n")[1].split(",")
11
12        # Check invalid symbol
13        if data[0] == "N/D":
14            print("Invalid stock symbol.")
15            return
16
17        print("Stock:", symbol.upper())
18        print("Date:", data[1])
19        print("Time:", data[2])
20        print("Open:", data[3])
```

Close Price: N/D
Stock: INVALID
Date: N/D
Time: N/D
Open: N/D
High: N/D
Low: N/D
Close Price: N/D

Explanation

This project retrieves real-time stock data for a company.

- User enters stock symbol (e.g., AAPL).
- The program fetches stock data from API.
- Displays:
 - Stock name
 - Current price
 - High & Low
 - Percentage change